



ALLIANCE
UNIVERSITY
Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Comparing of Deep Learning Architecture

Bachelor of Computer Application
Semester – V

Sub: Deep Learning

Topic: Comparing of Deep Learning Architecture

By

Name: Laishram Ritikumar Singh

Reg no.: 2411021240040

Faculty Name: Nagasri. P

Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

1. Introduction

Deep Learning is a branch of Artificial Intelligence and Machine Learning that uses artificial neural networks with multiple processing layers to learn patterns from data. Different deep learning architectures are designed for different types of input and tasks. Feedforward Neural Networks (FNNs) are suitable for relatively static and structured data, Convolutional Neural Networks (CNNs) are especially effective for spatial data such as images, and Recurrent Neural Networks (RNNs) are designed to process sequential or time-dependent data. Understanding these architectures helps in selecting an appropriate model for a particular real-world problem.

2. Objectives

- To understand the basic architecture and working principles of Feedforward Neural Networks, CNNs, and RNNs.
- To identify suitable real-world applications for each architecture.
- To compare the architectures based on structure, input data, strengths, limitations, and applications.
- To understand why different neural network architectures are preferred for different types of problems.

3. Task 1 – Basic Architecture of Deep Learning Models

3.1 Feedforward Neural Network (FNN)

A Feedforward Neural Network is one of the simplest forms of artificial neural network. Information moves in one direction, from the input layer through one or more hidden layers to the output layer. There are no connections that send information backward to previous layers.

Basic structure:

- Input Layer – receives the input features.
- Hidden Layer(s) – transform the input using weighted connections, biases, and activation functions.
- Output Layer – produces the final prediction or classification result.

Each neuron calculates a weighted sum of its inputs and applies an activation function. During training, the weights are adjusted using backpropagation and an optimization algorithm to reduce the prediction error.

Example: For predicting house prices, the input may contain area, number of rooms, location-related numerical features, and age of the house. The network can learn the relationship between these features and the target price.

3.2 Convolutional Neural Network (CNN)

A Convolutional Neural Network is a deep learning architecture primarily designed for data with spatial structure. CNNs are widely used in computer vision because they can automatically learn features such as edges, textures, shapes, and objects from images.

Typical CNN structure:

- Input Layer – receives an image or other grid-like input.
- Convolutional Layers – apply filters to detect local patterns and features.
- Activation Function – commonly ReLU, which introduces non-linearity.
- Pooling Layers – reduce spatial dimensions while retaining important information.
- Fully Connected Layer(s) – combine learned features for final prediction.
- Output Layer – produces a class, score, or other required output.

The major advantage of CNNs is local connectivity and parameter sharing. A filter can detect the same feature at different locations in an image, making CNNs more efficient than treating every pixel as an independent feature.

3.3 Recurrent Neural Network (RNN)

A Recurrent Neural Network is designed for sequential data. Unlike a basic feedforward network, an RNN maintains a hidden state that carries information from previous time steps. This allows the network to consider earlier elements of a sequence while processing the current element.

Basic working:

- The network receives one element of a sequence at a time.
- The current input is combined with information from the previous hidden state.
- A new hidden state is produced and passed to the next time step.
- The output can be generated at every time step or at the end of the sequence, depending on the task.

Traditional RNNs can experience difficulties learning long-term dependencies because of vanishing or exploding gradients. LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) architectures were developed to handle such dependencies more effectively.

4. Task 2 – Applications of Each Architecture

4.1 Applications of Feedforward Neural Networks

- Classification of structured or tabular data.
- Regression and numerical prediction.
- Customer or business analytics.
- Credit risk and basic fraud prediction.
- Student performance prediction.
- House price and sales prediction.

4.2 Applications of CNNs

- Image classification.
- Face and object detection.
- Medical image analysis.
- Handwritten digit recognition.
- Autonomous vehicle perception.
- Image segmentation.
- Quality inspection in manufacturing.

4.3 Applications of RNNs

- Text and language sequence processing.
- Speech and audio processing.
- Time-series forecasting.
- Sentiment and sequence classification.
- Machine translation and language modeling.
- Predictive maintenance using sensor sequences.
- Stock, weather, or demand forecasting.

5. Task 3 – Comparison of FNN, CNN, and RNN

Feature	FNN	CNN	RNN
Main purpose	General prediction and classification	Spatial feature learning	Sequential/time-dependent learning
Typical input	Structured/tabular data	Images and grid-like data	Sequences and time-series data
Core operation	Weighted connections	Convolution and pooling	Recurrence and hidden state
Memory of previous input	No	No inherent sequence memory	Yes, through hidden state
Parameter sharing	Generally no spatial sharing	Yes, convolutional filters	Yes, across time steps
Strength	Simple and effective for structured data	Excellent spatial feature extraction	Captures relationships across sequence steps
Common limitation	Not specialized for spatial or sequential structure	Can require significant computation/data	Training can be difficult for long dependencies
Examples	Price prediction, tabular classification	Image recognition, object detection	Time-series prediction, text processing

6. Structural Comparison

The three architectures differ mainly in the way they represent and process information. An FNN treats the input as a set of features and passes information forward through layers. A CNN exploits spatial relationships by applying learnable filters over local regions. An RNN processes inputs in sequence and uses a hidden state to carry information from earlier steps.

7. Input Data Comparison

- FNN: Best suited to fixed-length, structured, numerical, or categorical feature vectors.
- CNN: Best suited to images and other data where nearby elements have meaningful spatial relationships.
- RNN: Best suited to ordered sequences such as text, speech, sensor readings, and time-series observations.

8. Advantages and Limitations

8.1 Feedforward Neural Networks

Advantages:

- Simple architecture and easier to understand.
- Works well with many structured-data problems.

- Can be used for both classification and regression.

Limitations:

- Does not naturally model spatial relationships.
- Does not maintain a memory of previous sequence elements.
- May require careful feature preparation for complex raw data.

8.2 Convolutional Neural Networks

Advantages:

- Automatically learns useful spatial features.
- Parameter sharing reduces the number of parameters compared with fully connected image models.
- Highly effective for image and vision tasks.

Limitations:

- Training can require substantial computational resources.
- Performance may depend on a sufficiently large and representative dataset.
- Not naturally designed for long sequential dependencies.

8.3 Recurrent Neural Networks

Advantages:

- Designed for sequential and time-dependent data.
- Can use information from previous time steps.
- Useful for variable-length sequences.

Limitations:

- Traditional RNNs may struggle with long-term dependencies.
- Sequential computation can make training slower than highly parallel architectures.
- LSTM or GRU variants are often preferred for more complex long-range sequence patterns.

9. Real-World Example

Consider a fitness application that provides personalized analysis. An FNN could predict a user's calorie requirement from structured inputs such as age, weight, height, activity level, and training frequency. A CNN could analyze exercise images or video frames to recognize body positions and movement patterns. An RNN could analyze a sequence of workout sessions, heart-rate measurements, or running pace over time to identify trends and support forecasting. This example shows why choosing the architecture according to the data type is important.

10. Which Architecture Should Be Used?

The choice of architecture depends mainly on the structure of the input data and the objective of the task.

1. Use an FNN when the problem mainly involves structured features and there is no important spatial or temporal relationship.
2. Use a CNN when spatial patterns are important, especially for images, video frames, or grid-like data.
3. Use an RNN when the order of observations matters and information from previous time steps can help predict the current or future output.

11. Conclusion

Feedforward Neural Networks, Convolutional Neural Networks, and Recurrent Neural Networks are important deep learning architectures, but each is designed around different data characteristics. FNNs are effective for general structured-data prediction, CNNs are specialized for learning spatial features, and RNNs are designed to capture patterns in sequential data. Understanding their structures, strengths, limitations, and applications allows developers and data scientists to select an appropriate model for a given real-world problem. Therefore, there is no single architecture that is best for every task; the most suitable choice depends on the nature of the input data and the desired output.

12. References

4. Goodfellow, I., Bengio, Y., & Courville, A. – Deep Learning, MIT Press.
5. LeCun, Y., Bengio, Y., & Hinton, G. – Deep learning, Nature.
6. S. Haykin – Neural Networks and Learning Machines.
7. Course notes and classroom materials on Deep Learning Architectures.